194.048 Data-intensive Computing (VU 2,0) 2026S

Assignment 2: Text Processing and Classification using Apache Spark

Valentin Tian, 12426893
Tobias Haider, 11833743
Sevket Emin Cengay, 12552926
Burak Kilic, 12551186
Hafizullah Zubair, 12352877

May 19, 2026

Abstract

In this assignment, a scalable text classification pipeline was developed using Apache Spark to classify Amazon product reviews into 22 predefined product categories. The dataset consisted of 78,829 reviews and was processed through a distributed machine learning workflow. The used approach combines text preprocessing, tokenization, stop-word removal, TF-IDF feature extraction, chi-square feature selection, normalization, and one-vs-rest linear SVM classification. In addition, chi-square feature selection was implemented both manually with Spark RDD transformations and as part of a Spark ML pipeline.

Contents

1 Introduction

In this assignment, we implemented a complete large-scale text classification pipeline using Apache Spark. The objective was to classify Amazon product reviews into their corresponding product categories based on the textual content of the reviews.

The assignment consisted of three parts. In Part 1, we implemented chi-square feature selection using low-level Spark RDD transformations. In Part 2, we constructed a Spark ML pipeline for tokenization, stop-word removal, TF-IDF feature extraction, and chi-square feature selection. In Part 3, we trained and evaluated a multi-class Support Vector Machine (SVM) classifier using a one-vs-rest strategy and performed hyperparameter tuning through grid search.

The dataset contains 78,829 Amazon reviews distributed across 22 product categories. Apache Spark was used to process the data efficiently and to scale the machine learning workflow across multiple nodes.

2 Problem Overview

The goal of this assignment was to build a scalable text classification system that assigns each Amazon review to one of 22 predefined product categories.

The main challenges of the problem are:

- Processing a relatively large collection of textual documents.
- Handling a high-dimensional and sparse feature space.
- Selecting the most informative terms.
- Optimizing classification performance through hyperparameter tuning.

The classification task was divided into the following stages:

1. Text preprocessing and tokenization
2. Stop-word removal
3. TF-IDF feature extraction
4. Chi-square feature selection
5. Multi-class SVM classification
6. Hyperparameter tuning and evaluation

The performance of all models was measured using the macro F1-score, which gives equal importance to all categories regardless of their size.

3 Methodology and Approach

3.1 Part 1: Chi-Square Feature Selection with RDDs

In the first part, we implemented chi-square feature selection using Spark RDD transformations. Each review was tokenized and converted into a set of unique terms after removing stop words. For every term-category pair, occurrence counts were computed and used to calculate chi-square scores. The top 75 terms for each category were selected and combined into a final vocabulary.

3.2 Part 2: TF-IDF and Feature Selection with Spark ML

In the second part, we implemented a feature extraction pipeline using Spark ML. The pipeline included:

- RegexTokenizer
- StopWordsRemover
- CountVectorizer
- IDF
- StringIndexer
- ChiSqSelector

This pipeline selected the 2,000 most discriminative terms based on chi-square statistics.

3.3 Part 3: Multi-Class Text Classification

In the third part, we extended the pipeline by adding:

- Normalizer
- LinearSVC
- OneVsRest

The dataset was split into:

- 60% training
- 20% validation
- 20% test

The following hyperparameters were explored:

- Number of selected features: 2000, 500
- Regularization parameter (`regParam`): 0.01, 0.1, 1.0
- Standardization: True, False
- Maximum iterations (`maxIter`): 50, 100

This resulted in:

$$2 \times 3 \times 2 \times 2 = 24$$

different model configurations.

3.4 Pipeline Diagram

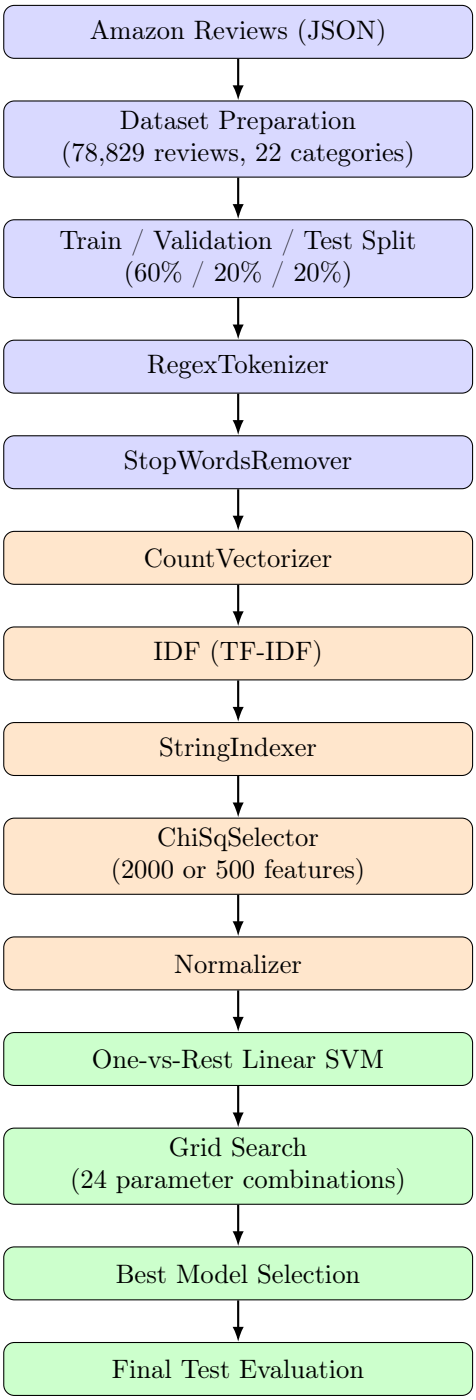

Figure 1: Overall text classification pipeline implemented in Apache Spark.

4 Results

4.1 Dataset Statistics

The dataset contained 78,829 reviews and 22 product categories. After random splitting, the resulting subsets were as follows.

Subset	Number of Reviews
Training	47,552
Validation	15,685
Test	15,592

Table 1: Dataset split statistics.

4.2 Intermediate Outputs

Part 1 Output. The RDD output is almost identical to the output from Assignment 1. Both files contain the same category-wise structure with 75 chi-square ranked terms per category and a joined terms dictionary at the bottom. The chi-square values and rankings are generally the same, with only minor formatting differences such as trailing zeros in floating-point values. A few differences occur only among the lowest-ranked terms. These small deviations are likely caused by tie handling, ordering of equally scored terms or implementation-specific sorting behavior.

Part 2 Output. The DataFrame output differs substantially from the Assignment 1 output because it solves a different feature selection task. Assignment 1 selects the top chi-square terms separately for each category, while Part 2 from the Assignment 2 selects 2000 terms overall across all categories using Spark's `ChiSqSelector`. Therefore, `output_ds.txt` is a single global list of selected terms rather than a category-wise list with chi-square scores.

4.3 Grid Search Results

A total of 24 parameter combinations were evaluated. Table 2 summarizes the best-performing settings.

Features	regParam	Standardization	maxIter	Validation F1
2000	0.01	True	50	0.6629
2000	0.01	True	100	0.6629
2000	0.10	True	50	0.6569
2000	0.10	True	100	0.6558
500	0.01	True	50	0.5561
500	0.01	True	100	0.5543

Table 2: Representative grid search results.

4.4 Best Model

The highest validation F1-score was obtained with the following hyperparameters.

Parameter	Value
Number of Selected Features	2000
regParam	0.01
Standardization	True
maxIter	100
Validation F1	0.6629

Table 3: Best model hyperparameters.

4.5 Final Test Performance

The selected model was evaluated on the independent test set.

Metric	Score
Validation F1	0.6629
Test F1	0.6600

Table 4: Final model performance.

4.6 Interpretation of Results

The experiments led to several important observations:

1. **Feature dimensionality strongly affects performance.** Reducing the feature set from 2,000 to 500 terms decreased the validation F1-score from approximately 0.66 to 0.56.
2. **Standardization is essential.** When standardization was disabled, the F1-score dropped substantially, often by more than 0.10.
3. **Lower regularization performed best.** The best results were obtained with `regParam = 0.01`.
4. **Additional iterations did not improve performance.** The results for 50 and 100 iterations were nearly identical, indicating that the optimization converged before reaching 50 iterations.

5 Conclusion

In this assignment, we implemented a scalable end-to-end text classification system using Apache Spark. The pipeline combined TF-IDF feature extraction, chi-square feature selection, and a one-vs-rest linear Support Vector Machine classifier.

The best model used 2,000 selected features, a regularization parameter of 0.01, enabled standardization, and achieved a validation F1-score of 0.6629 and a test F1-score of 0.6600.

The experiments showed that feature dimensionality and standardization are critical factors in classification performance. Reducing the number of features from 2,000 to 500 led to a significant

drop in accuracy, while increasing the maximum number of iterations beyond 50 provided no measurable benefit.

The close agreement between validation and test performance indicates that the selected model generalizes well to unseen data and does not exhibit significant overfitting.

Overall, this assignment demonstrated the effectiveness of Apache Spark for large-scale text analytics and highlighted the importance of efficient preprocessing, feature selection, and hyperparameter tuning in building robust machine learning models.